

FULL STACK DEVELOPMENT

FASE 5 | AULA 05 -

PATTERNS NO DESENVOLVIMENTO DE MICROSSERVIÇOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS	14

EXEMPLO

O QUE VEM POR AÍ?

Nesta aula, exploraremos padrões de design comuns e antipatterns no desenvolvimento de microserviços. Entenderemos como aplicar padrões como Service Discovery, API Gateway, Circuit Breaker, Saga, Fallback/Retry, Estrangulamento e Agregador, já que isso pode melhorar significativamente a resiliência, a escalabilidade e a eficiência dos sistemas. Também vamos discutir práticas a serem evitadas, conhecidas como antipatterns, que podem comprometer a modularidade e a performance dos microserviços. Por fim, veremos exemplos práticos de implementação desses padrões em empresas líderes de mercado, como Netflix, Spotify e Amazon.

HANDS ON

Nesta aula prática, serão apresentados os principais padrões de design utilizados em arquiteturas de microserviços, destacando a importância de cada um para garantir a resiliência, escalabilidade e eficiência do sistema. Vamos explorar como implementar padrões como Service Discovery, API Gateway, Circuit Breaker, Saga, Fallback/Retry, Estrangulamento e Agregador em projetos reais e discutir os antipatterns que devem ser evitados para garantir a eficiência e a manutenção do sistema.



SAIBA MAIS

Os padrões de design (design patterns) são soluções comprovadas para problemas recorrentes no desenvolvimento de software. Em arquiteturas de microsserviços, alguns padrões são amplamente utilizados para lidar com desafios específicos, melhorar a resiliência, a escalabilidade e a eficiência dos sistemas.

Um dos padrões disponíveis é o **Service Discovery**, que permite que os microsserviços encontrem e se comuniquem uns com os outros dinamicamente. Em um ambiente de microsserviços, onde instâncias de serviços podem escalar horizontalmente, serem atualizadas ou movidas entre diferentes servidores, é crucial que os serviços saibam onde encontrar uns aos outros. O Service Discovery resolve este problema permitindo que os serviços registrem suas localizações em um registro central e recuperem essas localizações quando necessário.

Ele pode ser implementado de forma client-side, onde o cliente usa uma biblioteca de descoberta para localizar instâncias de serviço, ou server-side, onde um load balancer ou gateway gerencia a descoberta e roteamento. Ferramentas comuns para implementação de Service Discovery incluem Consul, Eureka e Zookeeper. Esses sistemas monitoram as instâncias de serviço e atualizam o registro de descoberta conforme as instâncias são adicionadas ou removidas.

Os benefícios do Service Discovery incluem:

- **Autonomia dos serviços:** cada serviço pode ser independente e descobrir outros serviços sem a necessidade de configuração manual.
- **Escalabilidade:** permite a adição e remoção dinâmica de instâncias de serviço, facilitando a escalabilidade horizontal.
- **Resiliência:** ajuda a garantir que os serviços sejam sempre acessíveis, mesmo em face de falhas de instância.

Por exemplo, em uma aplicação de e-commerce, o serviço de carrinho de compras usa Service Discovery para localizar o serviço de catálogo de produtos, garantindo que sempre se conecte à instância correta e mais eficiente disponível. Isso assegura que as atualizações no catálogo de produtos sejam refletidas imediatamente no serviço de carrinho de compras.

Outro design pattern importante é o **API Gateway**, que atua como um ponto de entrada único para todos os clientes que acessam os microserviços. Ele roteia solicitações para os serviços apropriados, gerencia autenticação, autorização, limitação de taxa e pode realizar transformações de dados. Este padrão resolve vários problemas associados à comunicação direta entre clientes e microserviços, como gerenciamento de segurança e complexidade de integração.

Com o API Gateway, os clientes enviam solicitações para um endpoint único, que depois distribui essas solicitações para os microserviços apropriados. Isso simplifica a arquitetura do cliente e centraliza preocupações transversais como autenticação, logging e limitação de taxa. Ferramentas populares incluem Kong, NGINX e Amazon API Gateway.

Os benefícios do API Gateway incluem:

- **Centralização da segurança:** implementa autenticação e autorização de forma centralizada.
- **Roteamento inteligente:** direciona as solicitações para os serviços apropriados, gerenciando a carga e melhorando o desempenho.
- **Transformação de dados:** converte formatos de dados conforme necessário para integrar diferentes serviços.

Pensando em um cenário real, uma plataforma de streaming poderia utilizar um API Gateway para gerenciar solicitações de usuários para diferentes serviços, como reprodução de vídeos, gerenciamento de contas e recomendações. Isso facilitaria a implementação e manutenção do sistema, garantindo segurança e eficiência na comunicação entre os componentes.

Continuando a nossa jornada de aprendizado sobre design patterns, também temos o **Circuit Breaker**, que protege os serviços de falhas em cascata ao detectar falhas repetidas e abrir o circuito, impedindo novas tentativas de chamada ao serviço problemático até que ele se recupere. Em sistemas distribuídos, falhas em um serviço podem rapidamente sobrecarregar outros serviços dependentes, causando um efeito dominó que pode levar todo o sistema a falhar.

Implementar um Circuit Breaker envolve monitorar as chamadas de serviço e abrir o circuito quando um certo número de falhas consecutivas é detectado. Enquanto

o circuito está aberto, as chamadas para o serviço problemático são automaticamente falhadas, permitindo que o sistema se recupere. Uma vez que o serviço é restaurado, o circuito pode ser fechado, permitindo que as chamadas normais sejam retomadas. Ferramentas comuns para implementação incluem Netflix Hystrix e Resilience4j.

Os benefícios do Circuit Breaker incluem:

- **Resiliência:** protege o sistema contra falhas em cascata, melhorando a estabilidade.
- **Recuperação automática:** monitora o estado do serviço e restaura o circuito automaticamente quando o serviço se recupera.
- **Visibilidade:** fornece insights sobre a saúde dos serviços por meio de monitoramento contínuo.

Por exemplo, em um sistema de pagamento on-line, o Circuit Breaker pode interromper tentativas de conexão a um serviço de pagamento externo se ele estiver indisponível, evitando sobrecarga e falhas adicionais. Isso garante que o sistema permaneça funcional e que os usuários recebam respostas rápidas, mesmo em caso de falhas em serviços externos.

Já o padrão **Saga** é utilizado para gerenciar transações distribuídas em sistemas de microsserviços, onde cada passo de uma transação é gerenciado por um serviço separado e a coordenação é feita por mensagens. Em vez de uma transação única (como em sistemas monolíticos), as sagas permitem que cada serviço execute uma transação local e publique um evento ou mensagem para desencadear o próximo passo na cadeia.

Se qualquer passo da saga falhar, serviços subsequentes executam ações compensatórias para reverter as mudanças feitas pelos passos anteriores. Isso garante que o sistema mantenha um estado consistente, mesmo em face de falhas. A abordagem de sagas é particularmente útil em sistemas financeiros e de e-commerce, onde operações complexas e de longa duração são comuns.

Os benefícios do padrão Saga incluem:

- **Consistência distribuída:** garante que todas as partes da transação sejam consistentes, mesmo em sistemas distribuídos.

- **Resiliência:** permite a execução de ações compensatórias para reverter mudanças em caso de falhas.
- **Modularidade:** cada serviço gerencia sua própria parte da transação, facilitando a manutenção e escalabilidade.

Também temos o padrão **Fallback e Retry**, que trata da necessidade de lidar com falhas temporárias e intermitentes em sistemas distribuídos. Quando um serviço falha, o padrão Retry tenta a operação novamente após um intervalo definido, enquanto o padrão Fallback fornece uma alternativa ou resposta padrão quando todas as tentativas de Retry falham.

O Retry é especialmente útil para lidar com falhas temporárias de rede ou sobrecarga de serviço. Implementar Retry envolve definir políticas para o número de tentativas e os intervalos entre elas. Por outro lado, o Fallback fornece uma maneira de degradar graciosamente a funcionalidade em vez de falhar completamente, o que pode envolver retornar dados em cache, uma mensagem amigável ao usuário ou uma versão reduzida do serviço.

Os benefícios dos padrões Fallback e Retry incluem:

- **Resiliência:** garante que o sistema possa se recuperar de falhas temporárias e continue operando.
- **Experiência do usuário:** fornece alternativas para garantir que os usuários recebam respostas, mesmo quando ocorrem falhas.
- **Flexibilidade:** permite definir políticas específicas de retry e fallback para diferentes serviços.

Ferramentas como Resilience4j facilitam a implementação de padrões de Retry e Fallback, garantindo que o sistema permaneça resiliente em face de falhas. Por exemplo, em um serviço de recomendação de produtos, se o serviço de recomendação estiver indisponível, o sistema pode usar um conjunto de recomendações pré-calculadas ou populares como fallback.

Continuando nosso tópico, o padrão de **Estrangulamento** é utilizado para migrar sistemas monolíticos para uma arquitetura de microsserviços de forma incremental. Em vez de reescrever toda a aplicação de uma vez, o sistema existente

é gradualmente substituído por novos microserviços. O nome "estrangulamento" vem da ideia de que o novo sistema "estrangula" lentamente o antigo.

O processo começa com a identificação de partes da aplicação monolítica que podem ser isoladas e substituídas por serviços independentes. À medida que novos microserviços são desenvolvidos, eles substituem funcionalidades específicas do monolito, que eventualmente é desativado. Isso permite uma migração controlada e reduz o risco de falhas catastróficas.

Os benefícios do padrão Estrangulamento incluem:

- **Migração incremental:** permite a migração gradual de sistemas monolíticos para microserviços, minimizando o risco de falhas.
- **Menor impacto:** reduz o impacto nas operações diárias, permitindo que partes do sistema sejam substituídas sem interromper o serviço.
- **Facilidade de teste:** novos serviços podem ser testados e implementados individualmente, facilitando a identificação e correção de problemas.

Como exemplo temos uma aplicação legada de e-commerce, onde o serviço de pagamentos é reescrito como um microserviço independente. À medida que novos serviços, como catálogo de produtos e carrinho de compras, são desenvolvidos, o monolito original é gradualmente desativado.

Outro padrão é o **Agregador**, utilizado para combinar dados de vários serviços e fornecer uma resposta consolidada ao cliente. Em arquiteturas de microserviços, onde os dados podem estar espalhados por vários serviços, o Agregador atua como um intermediário que coleta, processa e combina informações antes de retornar uma resposta.

O Agregador pode ser implementado como um serviço independente ou como parte do API Gateway. Ele faz chamadas a múltiplos serviços, coleta os dados necessários e os combina em uma única resposta, simplificando a interação do cliente com o sistema. Embora similar ao padrão de design backend-for-frontend (BFF), um agregador é mais genérico e não especificamente utilizado para telas.

Os benefícios do padrão Agregador incluem:

- **Simplificação da interação:** reduz a complexidade para o cliente, consolidando múltiplas respostas em uma única.

- **Desempenho melhorado:** pode otimizar chamadas de serviço e reduzir a latência ao combinar dados em um único ponto.
- **Flexibilidade:** permite adicionar ou modificar serviços sem impactar diretamente os clientes, desde que o Agregador mantenha a interface.

Por exemplo, em uma aplicação de saúde, o serviço de agregação pode combinar dados de serviços de gerenciamento de pacientes, histórico médico e agendamento de consultas para fornecer uma visão completa do perfil de um paciente em uma única resposta.

Antipatterns

Ao desenvolver microsserviços, é importante estar ciente dos antipatterns — práticas que podem levar a problemas de manutenção, desempenho e escalabilidade. Evitar esses antipatterns ajuda a garantir a eficiência e a resiliência do sistema.

“Serviços gordos” (“Fat Services”)

Criar serviços que são muito grandes e complexos, agindo quase como um monolito, prejudica a modularidade e a escalabilidade dos microsserviços. Isso acontece quando um serviço assume muitas responsabilidades ou quando funcionalidades relacionadas são agrupadas sem uma clara separação de preocupações.

Para evitar “serviços gordos”, é crucial definir limites claros para cada serviço, mantendo-os pequenos e focados em uma única responsabilidade. Essa abordagem facilita a manutenção, a escalabilidade e permite que diferentes equipes trabalhem em serviços separados sem interferências.

Chamada remota excessiva (Chatty APIs)

Ter APIs que requerem muitas chamadas remotas para completar uma operação resulta em alta latência e baixa performance. Isso pode ocorrer quando os dados são distribuídos entre muitos serviços, e cada operação precisa reunir informações de várias fontes.

Agrupar dados relacionados em menos chamadas ou utilizar agregadores de dados pode ajudar a mitigar esse problema, e o design cuidadoso de contratos de API

e a adoção de padrões como CQRS podem reduzir a necessidade de múltiplas chamadas e melhorar a eficiência do sistema.

Bancos de dados compartilhados

Permitir que múltiplos serviços acessem diretamente o mesmo banco de dados cria um acoplamento forte, dificultando a escalabilidade e a independência dos serviços. Isso também aumenta a complexidade de gerenciamento de esquema e pode levar a problemas de concorrência.

Cada serviço deve possuir seu próprio banco de dados e comunicar-se com outros serviços via APIs ou eventos. Essa prática promove a independência dos serviços e facilita a escalabilidade e manutenção do sistema.

MERCADO, CASES E TENDÊNCIAS

Implementação de Service Discovery na Netflix

A Netflix utiliza Eureka para Service Discovery, permitindo que seus inúmeros microserviços se localizem e possam se comunicar dinamicamente. Isso é crucial para a escalabilidade da Netflix, onde novos serviços são constantemente implantados, possuem a necessidade de serem registrados e de serem descobertos automaticamente.

Circuit Breaker na Amazon

A Amazon implementa o padrão Circuit Breaker usando Resilience4j para garantir que falhas em serviços externos não causem um efeito em cascata em seu sistema. Isso protege a integridade do sistema e melhora a experiência do usuário durante períodos de falha.

Saga na Uber

A Uber utiliza o padrão Saga para gerenciar transações distribuídas complexas, como reservas de corridas. Se um serviço falhar durante a reserva, ações compensatórias são executadas para garantir que todas as partes da transação sejam consistentes e nenhuma operação seja deixada incompleta.

Fallback e Retry no Google

O Google implementa padrões de Fallback e Retry para garantir que seus serviços de pesquisa e publicidade permaneçam operacionais mesmo em face de falhas temporárias. Quando um serviço de anúncio falha, o sistema tenta novamente após um intervalo e, se todas as tentativas falharem, usa anúncios em cache como fallback.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os principais padrões de design utilizados no desenvolvimento de microsserviços, incluindo Service Discovery, API Gateway, Circuit Breaker, Saga, Fallback/Retry, Estrangulamento e Agregador. Discutimos como esses padrões ajudam a resolver desafios comuns em arquiteturas distribuídas, melhorando a resiliência, a escalabilidade e a eficiência dos sistemas. Também abordamos os antipatterns, práticas que devem ser evitadas para prevenir problemas de manutenção e desempenho. Por fim, analisamos exemplos práticos de como empresas líderes implementaram esses padrões em seus sistemas, proporcionando uma compreensão prática e aplicada dos conceitos discutidos.

REFERÊNCIAS

CHANG, A. Refactoring Legacy Code with the Strangler Fig Pattern. **Shopify Engineering**, 2020. Disponível em: <https://shopify.engineering/refactoring-legacy-code-strangler-fig-pattern/>. Acesso em: 24 set. 2024.

CIRCUIT Breaker Pattern. **AWS**, [s.d.]. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>. Acesso em: 24 set. 2024.

DIACONU, A. 8 Fallacies of Distributed Computing. **Ably**, 2022. Disponível em: <https://ably.com/blog/8-fallacies-of-distributed-computing>. Acesso em: 24 set. 2024.

EUREKA at a Glance. **Github**, 2014. Disponível em: <https://github.com/Netflix/eureka/wiki/Eureka-at-a-glance>. Acesso em: 24 set. 2024.

FOWLER, M. **Patterns of Enterprise Application Architecture**. [s.l.]: Addison-Wesley Professional, 2002.

KAPPAGANTULA, S. Everything You Need To Know About Microservices Design Patterns. **Edureka**, 2024. Disponível em: <https://www.edureka.co/blog/microservices-design-patterns>. Acesso em: 24 set. 2024.

NEWMAN, S. **Building Microservices. 2nd ed.** [s.l.]: O'Reilly Media, 2021.

ROSE, J. Introducing Domain-Oriented Microservice Architecture. **Uber Blog**, 2021. Disponível em: <https://www.uber.com/en-BR/blog/microservice-architecture/>. Acesso em: 24 set. 2024.

PALAVRAS-CHAVE

Banco de dados, Arquitetura de Software, Microserviços, Circuit Breaker, Saga, Fallback, Retry, Estrangulamento, Agregador.

EMSE

POSTECH